학습 노트: store 노드 — PostgreSQL + BigQuery 이중 적재

작성일: 2026-05-24 관련 파일: `dev2/nodes/store.py` (구현 예정), `dev2/db.py`, `dev2/schema.py`
복습 예정: D+1 (5/25), D+3 (5/27), D+7 (5/31)

한 줄 요약

store 노드는 분석이 끝난 이슈 목록을 "지금 바로 쓰는 운영 DB(PostgreSQL)"와 "나중에 통계 내는 분석 창고(BigQuery)" 두 곳에 나눠 저장하고, Dev4에게 넘길 표준 양식(Output Contract A)을 조립하는 파이프라인의 마지막 단계다.

---

1. 왜 PostgreSQL + BigQuery 이중 적재인가?

비유

편의점(PostgreSQL)과 대형 물류창고(BigQuery)의 차이. - 편의점: 손님이 오면 즉시 한두 개 꺼내준다. 빠르고 자주 쓴다. (실시간 조회/삽입) - 물류창고: 한 번에 수만 개를 쌓아두고 "이번 달 어느 상품이 제일 많이 나갔나"를 계산한다. (대용량 집계)

개념: **OLTP vs OLAP**

구분	PostgreSQL (OLTP)	BigQuery (OLAP)
목적	운영 트랜잭션	분석/집계
쿼리	"PR-001 이슈 보여줘" (행 단위)	"지난 3개월 Critical 추이" (수백만 행 집계)
속도	단건 INSERT/SELECT 빠름	대량 스캔 빠름 (열 기반 저장)
비용	항상 켜둠	스캔량 과금

왜 분리하나?

한 DB로 둘 다 하면 서로 방해한다. 분석용 무거운 집계 쿼리가 돌면 운영 DB가 느려져서, PR 리뷰 응답이 지연된다. 그래서 운영(OLTP)과 분석(OLAP)을 물리적으로 분리한다.

왜 **USE_MOCK_LLM=true** 시 **BigQuery skip?**

개발 중에는 분석 통계가 필요 없고, BigQuery는 클라우드 인증·과금이 걸린 외부 서비스라 로컬 개발자 환경에 부담이다. `BIGQUERY_PROJECT_ID`가 없으면 조용히 건너뛰어, DB 없이도 파이프라인이 끝까지 돈다 (CLAUDE.md §7 Mock 기반 병렬 개발).

---

2. store 노드의 역할 (파이프라인 마지막 노드)

```
enriched (list[dict])   ← llm_judge_node가 채운 최종 이슈들
    ↓ store_node
    ├── PostgreSQL: scan_executions → scan_issues → scan_summaries (순서 중요)
    └── BigQuery:   분석 DW 적재 (BIGQUERY_PROJECT_ID 없으면 skip)
    ↓
final_output (Output Contract A 형식 list[dict])  → Dev4로 전달
```

PostgreSQL 삽입 순서가 중요한 이유: `scan_issues`와 `scan_summaries`는 `scan_id`(FK)가 필요하므로, `insert_scan_execution`이 먼저 `scan_id`를 만들어야 한다.

---

3. Output Contract A 조립 방법

enriched 이슈는 출처가 섞여 있다. `_source` 필드로 분리해서 에이전트별 그룹을 만든다.

```
static_issues   = [i for i in enriched if i.get("_source") == "static"]
security_issues = [i for i in enriched if i.get("_source") == "security"]
```

각 그룹을 AgentOutput으로 만들고 `model_dump()`로 직렬화 → final_output 리스트에 담는다. 이렇게 하면 Dev4는 "static_analysis 결과", "security_scan 결과"를 따로 받는다 (CLAUDE.md §5-A).

---

4. BigQuery 적재 패턴 (실패해도 파이프라인 계속)

```python
def _load_bigquery(output: dict, payload: dict) -> None:
    project = os.getenv("BIGQUERY_PROJECT_ID", "")
    if not project:
        return  # 로컬 개발 스킵 (인증·과금 없음)
    try:
        from google.cloud import bigquery  # lazy import
        client = bigquery.Client(project=project)
        # rows 삽입 ...
    except Exception as e:
        logger.error("BigQuery 적재 실패: %s", e)
        # 로그만 — 파이프라인은 계속 (에러 재전파 안 함)
```

왜 **BigQuery** 실패가 전체 파이프라인을 막으면 안 되나?

BigQuery는 "나중에 통계 낼 부가 기능"이지, "PR 리뷰를 완성하는 필수 기능"이 아니다. 운영 DB(PostgreSQL) 저장이 성공했고 Output Contract가 만들어졌으면, Dev4는 이미 리뷰를 진행할 수 있다. 여기서 BigQuery 실패로 예외를 던지면, 분석용 부가 작업 때문에 정작 중요한 코드 리뷰가 통째로 죽는다. 이건 CLAUDE.md §12의 "Slack API 실패 → 로그만 남기고 무시" 와 같은 철학이다. 부가 서비스는 핵심 흐름을 막지 않는다.

---

5. CLAUDE.md §12 오류 처리 연결

대상	실패 시 처리	이유
PostgreSQL 삽입	에러 로그 + 재전파(raise)	운영 DB는 핵심. 저장 실패는 데이터 유실이므로 상위가 알아야 함 (CLAUDE.md §10-4 "에러 삼키지 않는다")
BigQuery 적재	로그만, 무시	분석용 부가 기능. 핵심 흐름 막으면 안 됨
`_build_output()` 검증	Pydantic AgentOutput 검증 후 직렬화	Dev4가 받는 계약 위반 방지

`_build_output()`에서 Pydantic 검증이 왜 필요한가?

enriched는 여러 노드를 거치며 자유롭게 dict 키가 추가/변형된 상태다. 그대로 Dev4에 넘기면 계약 위반(필드 누락, 잘못된 severity 값 등)이 런타임 한참 뒤에 터진다. AgentOutput으로 한 번 통과시키면, 잘못된 데이터가 store 노드 경계에서 즉시 걸러진다. "에이전트 출력은 Output Contract를 Pydantic으로 검증 후 전달" (CLAUDE.md §10-2)이 이 지점이다.

복습 퀴즈 (3문항)

1. (이해) PostgreSQL을 OLTP, BigQuery를 OLAP라고 부르는데, 한 DB로 둘 다 처리하면 어떤 문제가 생기는가?
2. (적용) BigQuery 적재가 실패했을 때 `raise`로 예외를 올리면 어떤 나쁜 일이 생기는가? PostgreSQL 삽입 실패와 처리를 다르게 하는 이유는?
3. (비교) `_build_output()`에서 enriched dict를 그대로 `final_output`에 넣지 않고 `AgentOutput(...).model_dump()`를 거치게 하는 이유는? 검증을 생략하면 언제 문제가 드러나는가?